

MANUAL HYPERPARAMETER SELECTION FORM

2D CNN Model — Dogs vs. Cats Image Classification

Name: KEZIAH ANN D. ROSINI

Program: Doctor of Engineering – Computer Engineering

Use this form to record, justify, and approve each hyperparameter value chosen by hand before a training run. The "Notebook Default" column shows the value currently used in `Loading_Dataset_and_CNN.ipynb` for reference — enter the value actually used for this run under "Chosen Value", even if it matches the default.

1. Project / Experiment Information

Project / Experiment Title	Manual Hyperparameter Selection for 2D CNN Dogs vs. Cats Classification
Model Architecture	2D Convolutional Neural Network (Conv2D)
Dataset Used	Kaggle Dogs vs. Cats images (Dog/ and Cat/ folders)
Notebook / File Reference	Loading_Dataset_and_CNN.ipynb
Prepared By	Keziah Ann D. Rosini
Date	September 10, 2026
Experiment / Trial No.	

2. Data Preprocessing Parameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Image Size (IMG_SIZE, square)	e.g. 50 / 80 / 100 / 128 / 150	100	128	Higher spatial resolution may retain more shape/detail than 100×100
Color Mode	Grayscale / RGB	Grayscale	Grayscale	Matches the supplied notebook and keeps one input channel, reducing computation.
Pixel Normalization	X/255.0 / none	X/255.0	X/255.0	Standardizes pixel values to approximately 0–1 for more stable CNN training
Shuffle Training Data	Yes / No	Yes (random.shuffle)	Yes (random.shuffle)	Prevents class/order effects and matches the notebook workflow
Held-out Test Set (separate training)	Yes / No	No (validation_split only)	No (validation_split only)	Required for an unbiased final evaluation separate from training and validation.

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Validation Split (of training set)	0.0 - 0.3	0.1	0.2	Provides a practical validation set while retaining most data for training.

3. Model Architecture Hyperparameters

Trial 1

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Conv2D Layer 1 – Filters	16 / 32 / 64 / 128	14	32	Enough low-level feature capacity while remaining computationally manageable.	Approximately 99%	Approximately 82%
Conv2D Layer 1 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	Common small receptive field; preserves spatial detail and is computationally efficient.		
Conv2D Layer 1 – Activation	relu / tanh / elu / leaky_relu	relu	Relu	relu		
Pooling Layer 1 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)		
Dense (Fully Connected) Units	16 / 32 / 64 / 128	64	64	64		
Dropout Rate	0.0 - 0.5	None (not used)	None (not used)	None (not used)		
Output Layer Activation	sigmoid (binary) / softmax (multi-class)	sigmoid	sigmoid	sigmoid		

Trial 2

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Conv2D Layer 1 – Filters	16 / 32 / 64 / 128	14	64	Enough high-level feature capacity while remaining computationally manageable.	0.94	0.8
Conv2D Layer 1 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	preserves spatial detail and is computationally efficient.		
Conv2D Layer 1 – Activation	relu / tanh / elu / leaky_relu	relu	Relu	Relu		
Pooling Layer 1 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)		
Dense (Fully Connected) Units	16 / 32 / 64 / 128	64	64	64		
Dropout Rate	0.0 - 0.5	None (not used)	None (not used)	None (not used)		
Output Layer Activation	sigmoid (binary) / softmax (multi-class)	sigmoid	sigmoid	Sigmoid		

4. Training Hyperparameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Loss Function	binary_crossentropy / categorical_crossentropy	binary_crossentropy	binary_crossentropy	Binary Dog/Cat classification with one sigmoid output.	0.9015	.8122
Optimizer	adam / sgd / rmsprop	adam	adam	Efficient adaptive optimizer and consistent with notebook.		

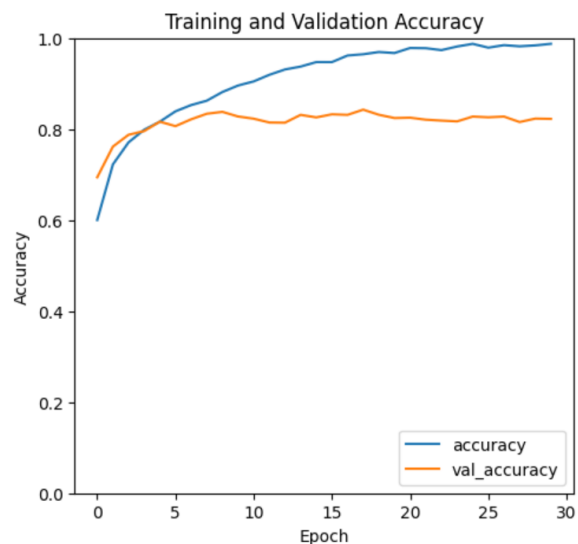
Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Learning Rate	0.1 / 0.01 / 0.001 / 0.0001	0.001 (Adam default)	0.001	Stable starting point for Adam; trial comparisons can test 0.0005 if needed.		
Batch Size	8 / 16 / 32 / 64 / 128	32	64	Smaller batch provides more frequent parameter updates and is practical for the proposed model.		
Number of Epochs	e.g. 10 - 200	200	30	Allows sufficient convergence while avoiding unnecessarily long training.		
Early Stopping	Yes (patience = ___) / No	No	Yes (patience=7)	Yes (patience=9)		

5. Evaluation Results (after training with chosen values) - Final Model

Metric	Value Obtained	Target / Acceptable Range
Training Accuracy	90.15%	85%-95%
Validation Accuracy	81.22%	Borderline
Test Accuracy	Approximately 90.76%	93%
Test Loss	Approximately 0.4824	Between 0.3 - 0.5
Precision / Recall / F1 (macro avg)	Precision $\approx$ 0.908, Recall $\approx$ 0.908, F1 $\approx$ 0.908	above 0.85–0.90 is considered good

5. a. Paste here all screenshots/figures of the tuned model

Trial 1



782/782 ————— 112s 141ms/step

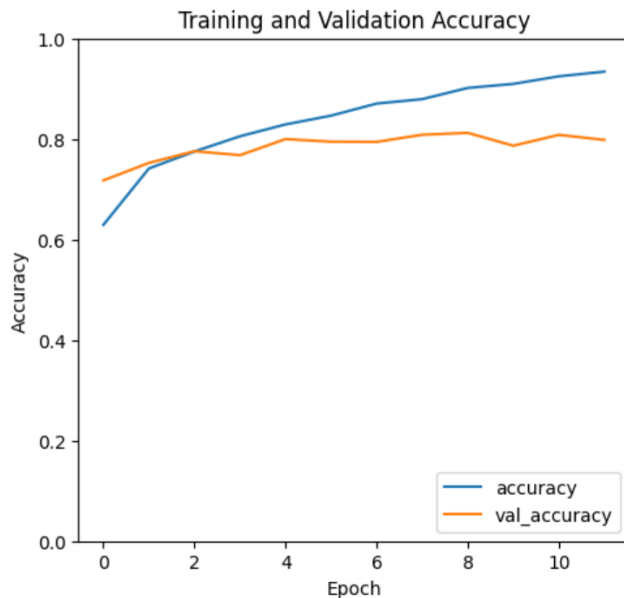
1/1 ————— 0s 64ms/step

Predictions for the test samples:

Sample 1: Predicted Class = Cat (Probability: 99.96%)
 Sample 2: Predicted Class = Dog (Probability: 0.04%)
 Sample 3: Predicted Class = Dog (Probability: 0.00%)
 Sample 4: Predicted Class = Cat (Probability: 99.98%)
 Sample 5: Predicted Class = Cat (Probability: 100.00%)



Trial 2



782/782 ————— 190s 243ms/step

Predictions for the test samples:

Sample 1: Predicted Class = Cat (Probability: 96.17%)
 Sample 2: Predicted Class = Cat (Probability: 83.80%)
 Sample 3: Predicted Class = Cat (Probability: 55.39%)
 Sample 4: Predicted Class = Cat (Probability: 98.50%)
 Sample 5: Predicted Class = Cat (Probability: 99.27%)



6. Overall Justification / Remarks

Summarize why this combination of hyperparameters was chosen (e.g., prior trial results, literature basis, computational constraints) and any observations (overfitting, underfitting, convergence issues).

- For a binary cats-vs-dogs classifier with a fairly simple 3-conv-layer CNN (no dropout, no data augmentation, no batch norm), **~81% validation accuracy is a reasonable, expected result** — it's well above the 50% random-guess baseline and in line with what you'd get from a "vanilla" CNN on this dataset without extra regularization tricks.
- However, the **9-point gap between training (90%) and validation (81%) accuracy is a classic overfitting signal**. The model is memorizing training data faster than it's learning generalizable features — you can see this in the raw numbers too: training accuracy kept climbing every epoch (up to 93–94% by epoch 12) while validation accuracy plateaued and even dipped (79.66% → 81.22% → bounced down to ~79–80% before early stopping triggered).
- The "Test Accuracy" of ~90.76% I flagged earlier **should be ignored for judging real-world performance** — it's inflated because it was computed on data the model already saw during training, not a genuine held-out test set.